



Over 750 broadband networks in the US

A new map showcases that more communities than ever are building their own broadband networks. <http://digit.in/750bb>



Gmail two-factor authentication

A Google engineer revealed that more than 90 percent of gmail users don't use two-factor authentication. <http://digit.in/gm2faca>

to fit in with the rest of your app's visual style, and it is open source.

Firebase Authentication SDK

- Authenticate users with their email addresses and passwords. Also handles sending password reset emails.
- Authenticate users by integrating with federated identity providers, like Google, Facebook, Twitter etc.
- SMS authentication.
- Connect your app's existing sign-in system to the Firebase Authentication SDK and gain access to Firebase Realtime Database and other Firebase services.
- Use Firebase features that require authentication without making users sign in first by creating temporary anonymous accounts. If the user chooses to sign up, you can upgrade the anonymous account to a regular account, so the user can continue where they left off.

How to implement?

FirebaseUI Auth

1. **Set up sign-in methods** - For email address and password or phone number sign-in and any federated identity providers you want to support, enable them in the Firebase console and complete any configuration required by the identity provider, such as setting your OAuth redirect URL.
2. **Customize the sign-in UI** - You can customize the sign-in UI by setting FirebaseUI options or fork the code on GitHub to customize the sign-in experience further.
3. **Use FirebaseUI to perform the sign-in flow** - Import the FirebaseUI library, specify the sign-in methods you want to support, and initiate the FirebaseUI sign-in flow.

Firebase Authentication SDK

1. **Set up sign-in methods** - Follow the same steps as for FirebaseUI Auth.
2. **Implement UI flows for your sign-in methods** - For email address and password sign-in, implement a flow that prompts users to type their email addresses and passwords. For

phone number sign-in, create a flow that prompts users for their phone number, and then for the code from the SMS message they receive. For federated sign-in, implement the flow required by each provider.

3. **Pass the user's credentials to the Firebase Authentication SDK** - Pass the user's email address and password or the OAuth token that was acquired from the federated identity provider to the Firebase Authentication SDK.

FIREBASE REALTIME DATABASE

It is a cloud-hosted NoSQL database. Data is stored as JSON and synchronized in Real Time to every connected client. When you build cross-platform apps with the iOS, Android, and JavaScript SDKs, all clients share one Real Time Database instance and automatically receive updates with newest data. This database also accommodates for your app growing in scale and popularity seamlessly.

Features

- **Realtime** - Instead of typical HTTP requests, the Firebase Realtime Database uses data synchronization—every time data changes, any connected device receives that update within milliseconds.
- **Offline** - Firebase apps remain responsive even when offline because the Firebase Realtime Database SDK persists your data to disk. Once connectivity is reestablished, the client device receives any changes it missed, synchronizing it with the current server state.
- **Accessible from Client Devices** - The Firebase Realtime Database can be accessed directly from a mobile device or web browser; there's no need for an application server. Security and data validation are available through the Firebase Realtime Database Security Rules, expression-based rules that are executed when data is read or written.
- **Scale across multiple databases** -

```
mAuth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener(this, new OnCompleteListener<AuthResult>() {
        @Override
        public void onComplete(@NonNull Task<AuthResult> task) {
            if (task.isSuccessful()) {
                // Sign in success, update UI with the signed-in user's information
                Log.d(TAG, "createUserWithEmail:success");
                FirebaseUser user = mAuth.getCurrentUser();
                updateUI(user);
            } else {
                // If sign in fails, display a message to the user
                Log.w(TAG, "createUserWithEmail:failure", task.getException());
                Toast.makeText(EmailPasswordActivity.this, "Authentication failed.",
                    Toast.LENGTH_SHORT).show();
                updateUI(null);
            }

            // ...
        }
    });
```

Sign up users

```
FirebaseUser user = FirebaseAuth.getInstance().getCurrentUser();
if (user != null) {
    // Name, email address, and profile photo URL
    String name = user.getDisplayName();
    String email = user.getEmail();
    Uri photoUrl = user.getPhotoUrl();

    // Check if user's email is verified
    boolean emailVerified = user.isEmailVerified();

    // The user's ID, unique to the Firebase project. Do NOT use this value to
    // authenticate with your backend server, if you have one. Use
    // FirebaseUser.getUid() instead.
    String uid = user.getUid();
}
```

Get logged in users